

Santiago Lawn Tennis Club — Santiago del Estero



Trabajo Práctico N.º 2

AP2 — Análisis · Diseño · Implementación · Pruebas
Base de Datos · Comunicaciones

Alumno	Fonzo, Ignacio
Fecha	Mayo de 2026
Materia	Seminario de Práctica
Repositorio	github.com/nachof9/SistRUGBY-SLTC

1. Introducción	3
1.1 Correcciones incorporadas del AP1	3
2. Etapa de Análisis	4
2.1 Diagrama de Clases de Análisis	4
2.2 Diagrama de Secuencia — CU01: Iniciar Sesión	5
2.3 Diagrama de Secuencia — CU05: Registrar Partido	6
3. Etapa de Diseño	7
3.1 Arquitectura del Sistema	7
3.2 Diagrama de Clases de Diseño	8
3.3 Diagrama de Componentes	9
3.4 Diagrama de Despliegue	10
4. Etapa de Implementación	11
4.1 Descripción del Prototipo	11
4.2 Estructura de Paquetes Java	11
4.3 Código Fuente Representativo	12
4.4 Repositorio GitHub	13
5. Etapa de Pruebas	14
5.1 Plan de Pruebas	14
5.2 Casos de Prueba	14
6. Definición de Base de Datos	16
6.1 Modelo Relacional Normalizado	16
6.2 Diagrama Entidad-Relación (DER)	17
6.3 Creación de Tablas (DDL)	18
6.4 Inserción de Registros (DML)	19
6.5 Consultas SQL	20

6.6 Eliminación de Registros	2
	1
7. Definiciones de Comunicación	22
7.1 Entorno de Red e Infraestructura Física	2
	2
7.2 Protocolos y Estándares de Comunicación	2
	3
8. Referencias Bibliográficas	24

1. Introducción

El presente documento corresponde al Trabajo Práctico N° 2 (AP2) de la materia Seminario de Práctica de la Licenciatura en Informática. Retoma el proyecto SistRUGBY-SLTC definido en AP1 y avanza en las fases de Elaboración y Construcción del Proceso Unificado de Desarrollo (PUD), incorporando las correcciones indicadas por la docente en la entrega anterior.

Se incorporan los siguientes artefactos: modelo de clases de análisis y diagramas de secuencia del sistema (Análisis); arquitectura detallada, diagrama de clases de diseño, componentes y despliegue (Diseño); descripción del prototipo con código Java representativo (Implementación); plan y casos de prueba (Pruebas); modelo relacional normalizado, DER, scripts DDL/DML (Base de Datos); y definiciones de comunicación del sistema.

Correcciones del AP1 incorporadas en esta entrega:

Observación docente	Acción correctiva aplicada
Mejorar el índice con alineación justificada y estilo formal	Índice reconstruido con tabla de contenidos formalmente estructurada y alineación consistente
Redactar definiciones del proyecto como objetivos	La sección de definiciones en AP1 fue revisada; en AP2 todos los objetivos de cada etapa se redactan en forma declarativa y medible
Incluir el actor en la redacción de los requerimientos	Las tablas de RF en AP1 fueron actualizadas; en AP2 se mantiene el patrón "El [Actor] debe poder..."
Los nombres de CU en el diagrama deben coincidir con los especificados en la tabla	Todos los diagramas de esta entrega usan exactamente los mismos identificadores y nombres que las tablas de CU en AP1

2. Etapa de Análisis

En la fase de Elaboración del PUD se profundiza el modelo de análisis comenzado en AP1. Los artefactos centrales de esta etapa son el diagrama de clases de análisis, que modela el dominio del problema en términos de clases conceptuales y sus relaciones, y los diagramas de secuencia del sistema, que describen la interacción entre actores y el sistema para los casos de uso de mayor criticidad (Larman, 2003).

2.1 Diagrama de Clases de Análisis

El diagrama identifica las clases del dominio del problema, sus atributos relevantes, operaciones principales y las asociaciones entre ellas. En este nivel de análisis no se detallan aspectos de implementación como DAOs o capas de persistencia; el foco está en el modelo conceptual del negocio (Jacobson et al., 1999).

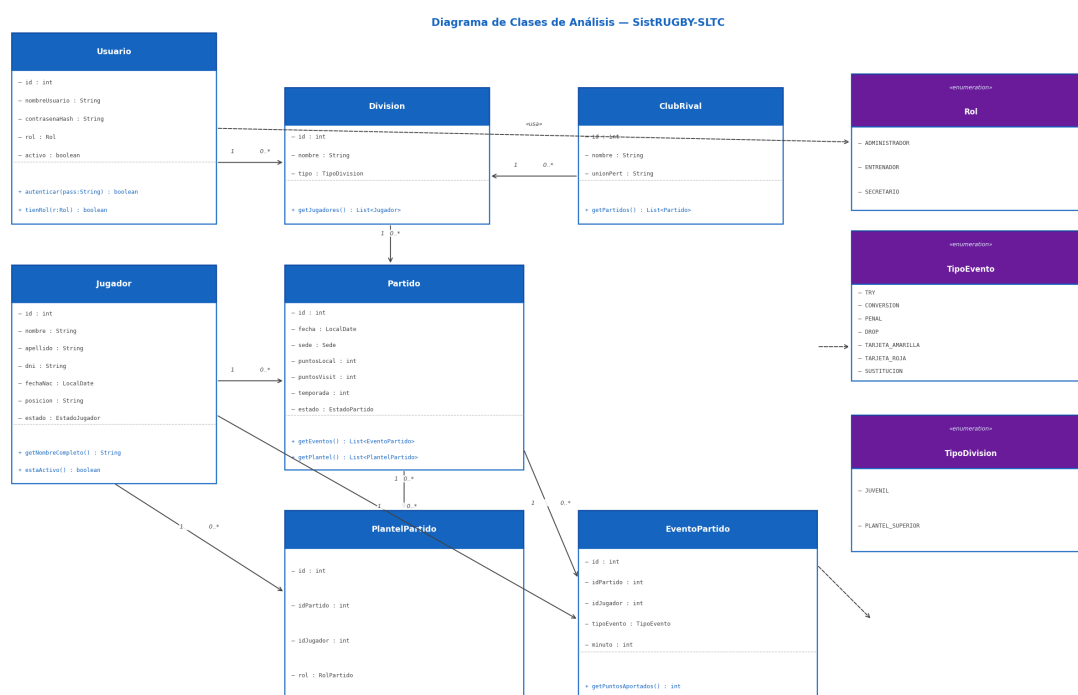


Figura 1 — Diagrama de Clases de Análisis — SistRUGBY-SLTC — Fase Elaboración / PUD

Las clases identificadas son: Usuario (gestión de acceso con roles), Jugador (padrón deportivo), Division (categorías M15-Primera), Partido (fixture competitivo), ClubRival (equipos de la USR), PlantelPartido (asociación N:M entre Partido y Jugador) y EventoPartido (registro de tries, tarjetas, etc.). Las enumeraciones Rol, TipoEvento y TipoDivision tipifican los valores discretos del dominio.

2.2 Diagrama de Secuencia del Sistema — CU01: Iniciar Sesión

El diagrama de secuencia del sistema (DSS) muestra los mensajes que intercambia el actor con el sistema como caja negra, para el flujo principal del caso de uso. A continuación se detalla el DSS del CU01, caso de uso de prioridad Alta que constituye la puerta de entrada al sistema.

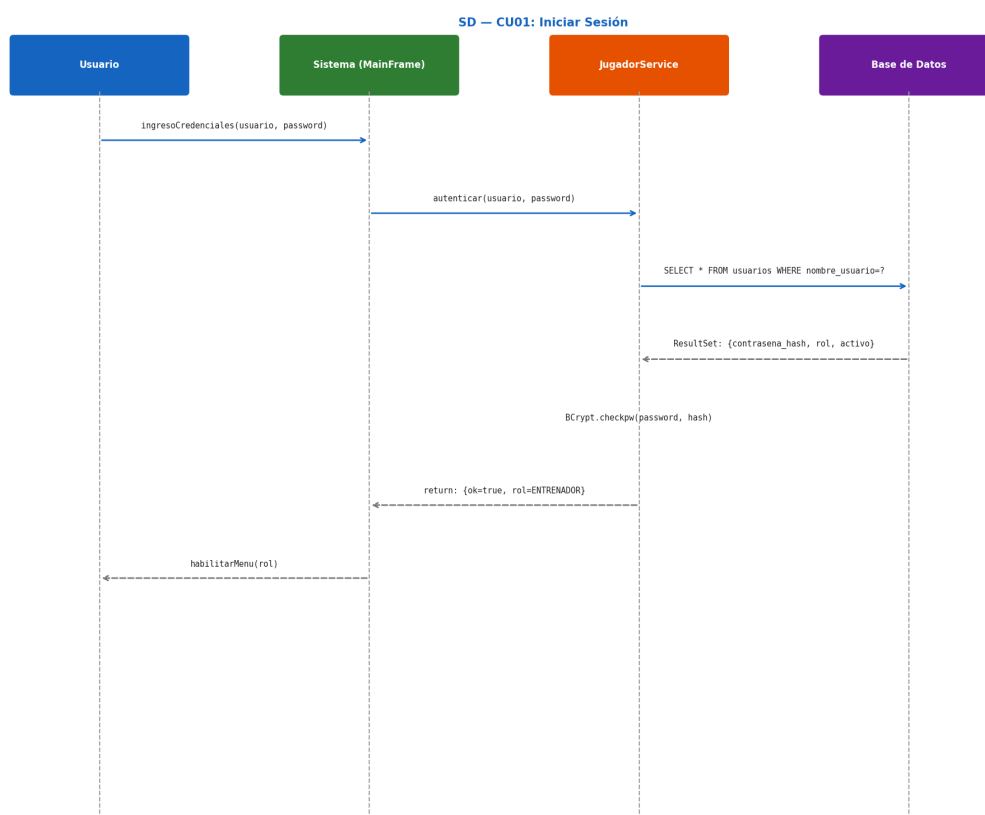


Figura 2 — Diagrama de Secuencia: CU01 Iniciar Sesión — SistRUGBY-SLTC

La validación de credenciales se realiza mediante bcrypt (BCrypt.checkpw), evitando comparar contraseñas en texto plano. Si la autenticación falla tres veces consecutivas, la cuenta se bloquea por 5 minutos (flujo alternativo documentado en CU01 — AP1). El rol retornado determina las funcionalidades habilitadas en el menú principal.

2.3 Diagrama de Secuencia del Sistema — CU05: Registrar Partido

El CU05 es el caso de uso de mayor complejidad operativa: integra tres capas del sistema (GUI, lógica de negocio y persistencia) y es precondition para los CU06 y CU07. El flujo alternativo 5a —que invoca CU11 por «extend» cuando el club rival no existe— no se detalla en este DSS por claridad, pero está documentado en AP1.

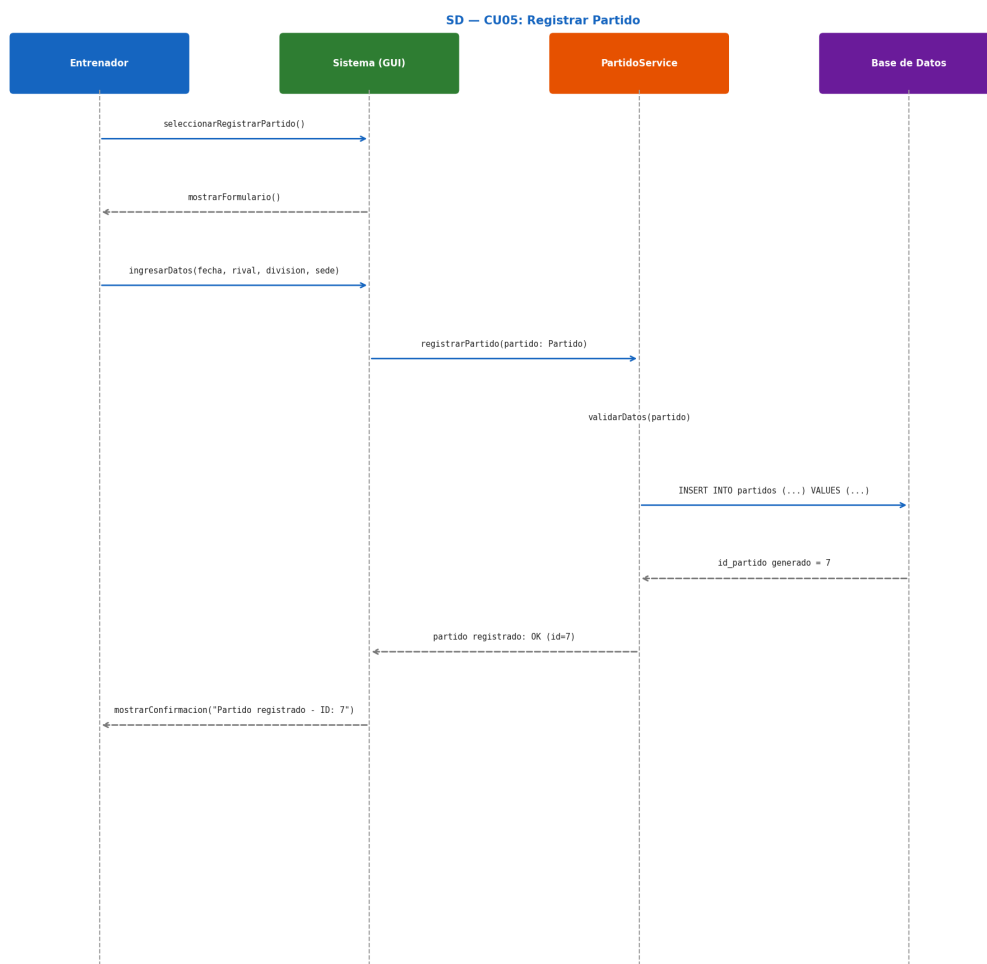


Figura 3 — Diagrama de Secuencia: CU05 Registrar Partido — SistRUGBY-SLTC

3. Etapa de Diseño

La etapa de diseño traduce el modelo de análisis en una solución técnica concreta, respetando la arquitectura de tres capas lógicas definida en AP1 y las restricciones tecnológicas del proyecto (Java SE 17, Swing, MySQL 8.0, JDBC).

3.1 Arquitectura del Sistema

SistRUGBY-SLTC adopta una arquitectura cliente-servidor de dos capas físicas (aplicación Java + servidor MySQL en la misma máquina o en red local), con tres capas lógicas:

Capa	Responsabilidad	Tecnología	Patrón aplicado
Presentación	Interfaz gráfica de usuario; captura y visualización de datos	Java Swing (JFrame, JPanel, JTable)	MVC (Vista)
Negocio	Reglas de negocio, validaciones, cálculo de estadísticas	Java SE 17 (clases de servicio)	Service Layer
Persistencia	Acceso y gestión de datos en la base de datos relacional	JDBC — MySQL Connector/J 8.3	DAO (Data Access Object)

El patrón DAO desacopla la lógica de negocio de la tecnología de persistencia, facilitando la mantenibilidad y el testing. Las interfaces DAO (JugadorDAO, PartidoDAO, etc.) permiten sustituir la implementación concreta sin modificar las capas superiores.

3.2 Diagrama de Clases de Diseño

El diagrama muestra la estructura de clases a nivel de implementación, organizada en cuatro paquetes Java: presentacion, negocio, persistencia y modelo. Las dependencias entre paquetes siguen estrictamente la dirección descendente: presentacion → negocio → persistencia → modelo.

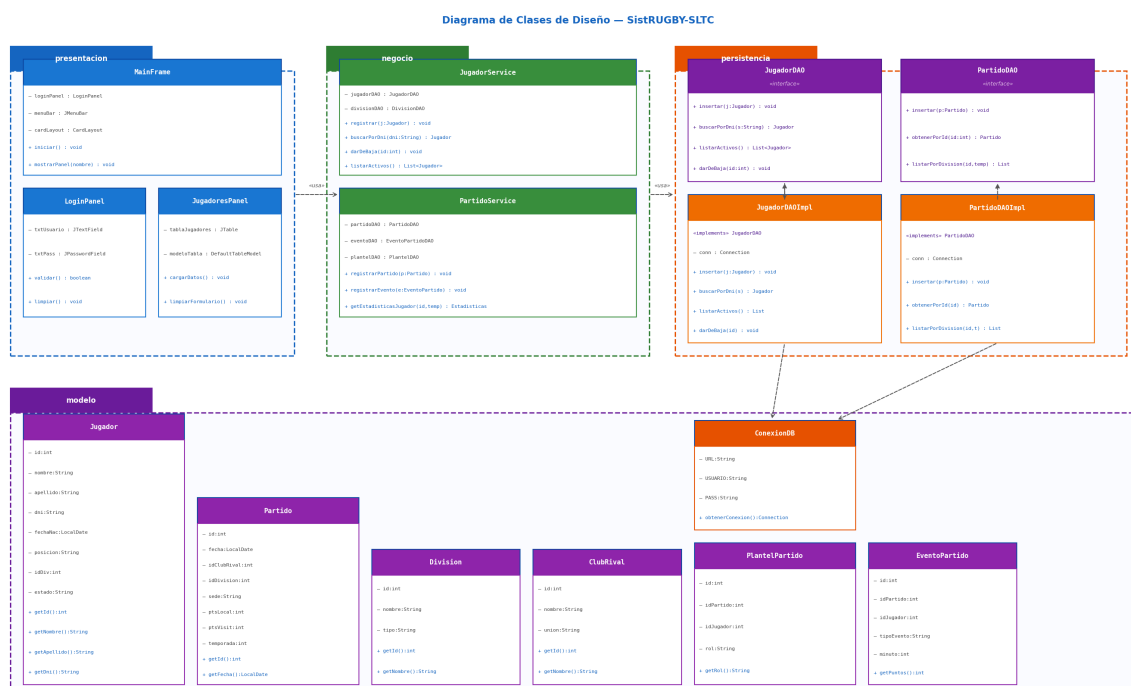


Figura 4 — Diagrama de Clases de Diseño — SistRUGBY-SLTC — Paquetes y relaciones

3.3 Diagrama de Componentes

El diagrama de componentes muestra los componentes lógicos del sistema, sus interfaces provistas y requeridas, y las dependencias entre ellos. El componente de base de datos (MySQL 8.0) se comunica con la Capa DAO exclusivamente a través de JDBC sobre TCP puerto 3306.

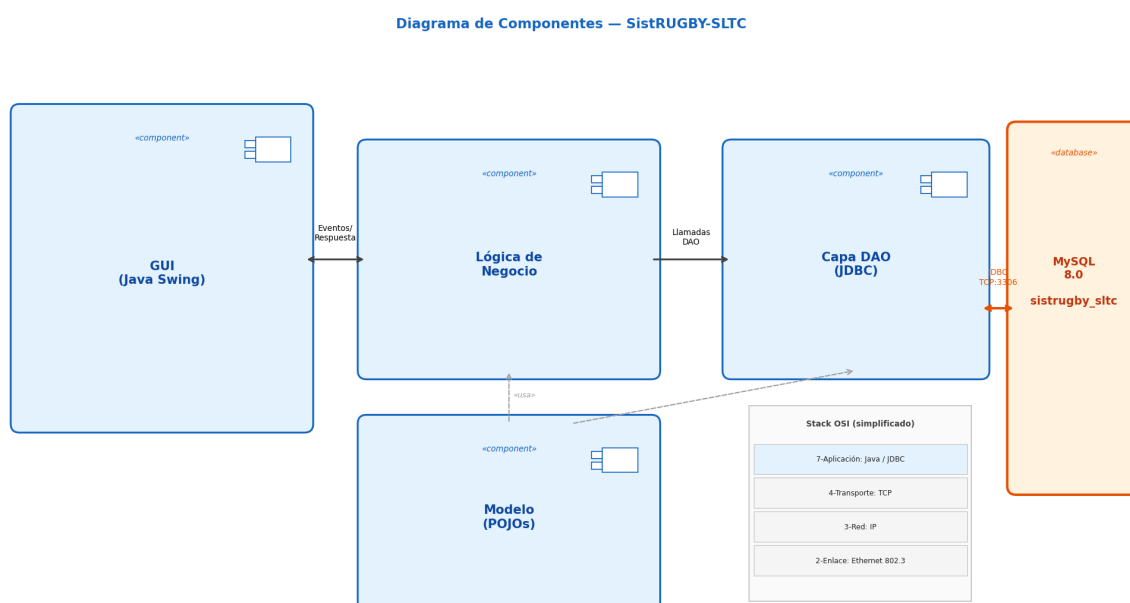


Figura 5 — Diagrama de Componentes — SistRUGBY-SLTC

3.4 Diagrama de Despliegue

El despliegue contempla una única máquina física (PC del club) que aloja tanto la JVM con la aplicación como el servidor MySQL. Esta topología simplifica la instalación y es adecuada para el volumen de datos y usuarios del SLTC. Una migración futura a servidor dedicado solo requeriría cambiar la cadena de conexión JDBC.

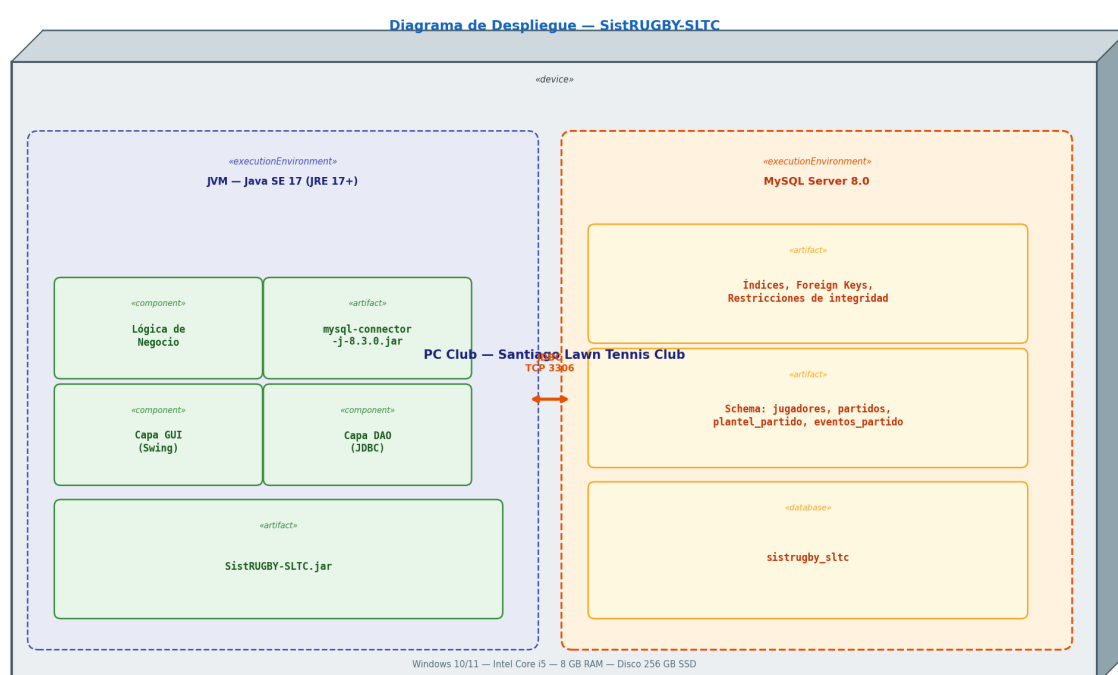


Figura 6 — Diagrama de Despliegue — SistRUGBY-SLTC

4. Etapa de Implementación

4.1 Descripción del Prototipo

En el marco del PUD, la fase de Construcción produce entregas incrementales. El prototipo actual (Sprint 1 y Sprint 2 según el plan de AP1) implementa los módulos de autenticación y gestión de jugadores, cubriendo los CU01, CU02, CU03 y CU04. El código completo y actualizado está disponible en el repositorio GitHub.

El prototipo es operacional: permite iniciar sesión con usuario/contraseña hasheada con bcrypt, listar jugadores activos en una tabla Swing y registrar nuevos jugadores validando DNI duplicado. La base de datos MySQL está inicializada con el script schema.sql y datos de prueba cargados con datos_iniciales.sql.

4.2 Estructura de Paquetes Java

Paquete	Clases principales	Responsabilidad
com.sltc.sistrugby.modelo	Jugador, Partido, Division, ClubRival, PlantelPartido, EventoPartido, Usuario	POJOs del dominio (atributos + getters/setters)
com.sltc.sistrugby.persistencia	ConexionDB	Singleton de conexión JDBC a MySQL
com.sltc.sistrugby.persistencia.dao	JugadorDAO, PartidoDAO, EventoPartidoDAO, PlantelDAO	Interfaces DAO (contrato de acceso a datos)
com.sltc.sistrugby.persistencia.impl	JugadorDAOImpl, PartidoDAOImpl, EventoPartidoDAOImpl	Implementaciones concretas JDBC con PreparedStatement
com.sltc.sistrugby.negocio	JugadorService, PartidoService	Lógica de negocio y validaciones, llama a los DAO
com.sltc.sistrugby.presentacion	MainFrame, LoginPanel, JugadoresPanel	GUI Swing: ventanas, paneles y formularios

4.3 Código Fuente Representativo

ConexionDB.java — Conexión JDBC

```

ConexionDB.java
package com.sltc.sistrugby.persistencia;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * Proveedor de conexiones JDBC a MySQL.
 * Patrón: Singleton + Factory Method.
 */
public class ConexionDB {
    private static final String URL =
        "jdbc:mysql://localhost:3306/sistrugby_sltc?useSSL=false&serverTimezone=UTC";
    private static final String USUARIO = "sistrugby_user";
    private static final String PASSWORD = System.getenv("DB_PASS"); // variable de entorno

    private ConexionDB() {}

    public static Connection obtenerConexion() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, PASSWORD);
    }
}

```

Jugador.java — Clase de Dominio (fragmento)

```

Jugador.java
package com.sltc.sistrugby.modelo;

import java.time.LocalDate;

public class Jugador {
    private int id;
    private String nombre;
    private String apellido;
    private String dni;
    private LocalDate fechaNacimiento;
    private String posicion;
    private int idDivision;
    private String estado; // "ACTIVO" | "INACTIVO"
    private LocalDate fechaAlta;

    public Jugador() {}

    public Jugador(String nombre, String apellido, String dni,
        LocalDate fechaNacimiento, String posicion, int idDivision) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.dni = dni;
        this.fechaNacimiento = fechaNacimiento;
        this.posicion = posicion;
        this.idDivision = idDivision;
        this.estado = "ACTIVO";
        this.fechaAlta = LocalDate.now();
    }

    public String getNombreCompleto() { return apellido + ", " + nombre; }
    public boolean estaActivo() { return "ACTIVO".equals(estado); }

    // Getters y setters omitidos por brevedad (ver repositorio GitHub)
}

```

JugadorDAOImpl.java — Capa de Persistencia (métodos clave)**JugadorDAOImpl.java**

```

package com.sltc.sistrugby.persistencia.impl;

import com.sltc.sistrugby.modelo.Jugador;
import com.sltc.sistrugby.persistencia.ConexionDB;
import com.sltc.sistrugby.persistencia.dao.JugadorDAO;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class JugadorDAOImpl implements JugadorDAO {

    @Override
    public void insertar(Jugador j) throws Exception {
        String sql = "INSERT INTO jugadores (nombre, apellido, dni, fecha_nacimiento, " +
            "posicion, id_division, estado, fecha_alta) VALUES (?, ?, ?, ?, ?, ?, 'ACTIVO', CURDATE())";
        try (Connection conn = ConexionDB.obtenerConexion();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, j.getNombre());
            ps.setString(2, j.getApellido());
            ps.setString(3, j.getDni());
            ps.setDate(4, Date.valueOf(j.getFechaNacimiento()));
            ps.setString(5, j.getPosicion());
            ps.setInt(6, j.getIdDivision());
            ps.executeUpdate();
        }
    }

    @Override
    public Jugador buscarPorDni(String dni) throws Exception {
        String sql = "SELECT * FROM jugadores WHERE dni = ?";
        try (Connection conn = ConexionDB.obtenerConexion();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, dni);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return mapear(rs);
        }
        return null;
    }

    @Override
    public void darDeBaja(int id) throws Exception {
        String sql = "UPDATE jugadores SET estado = 'INACTIVO' WHERE id_jugador = ?";
        try (Connection conn = ConexionDB.obtenerConexion();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }

    private Jugador mapear(ResultSet rs) throws SQLException {
        Jugador j = new Jugador();
        j.setId(rs.getInt("id_jugador"));
        j.setNombre(rs.getString("nombre"));
        j.setApellido(rs.getString("apellido"));
        j.setDni(rs.getString("dni"));
        j.setFechaNacimiento(rs.getDate("fecha_nacimiento").toLocalDate());
        j.setPosicion(rs.getString("posicion"));
        j.setIdDivision(rs.getInt("id_division"));
        j.setEstado(rs.getString("estado"));
        return j;
    }
}

```

}

4.4 Repositorio GitHub

El código fuente completo, los scripts SQL y la documentación técnica se encuentran disponibles en el repositorio público:

<https://github.com/nachof9/SistRUGBY-SLTC>

Directorio	Contenido
src/main/java/com/slrc/sistrugby/	Código fuente Java organizado en paquetes (modelo, persistencia, negocio, presentacion)
sql/schema.sql	Script DDL: CREATE DATABASE + CREATE TABLE con restricciones e índices
sql/datos_iniciales.sql	Script DML: INSERT de divisiones, clubes, usuarios y jugadores de prueba
docs/	Documentos AP1 y AP2, capturas de diagramas UML
README.md	Guía de instalación, requisitos del sistema y ejecución del prototipo

5. Etapa de Pruebas

5.1 Plan de Pruebas

El plan de pruebas cubre los módulos del prototipo correspondientes a Sprint 1 y 2. Se aplican pruebas funcionales de caja negra sobre los casos de uso implementados. El entorno de prueba es idéntico al de desarrollo: Java SE 17, MySQL 8.0, Windows 11.

Atributo	Detalle
Alcance	CU01 (Iniciar sesión), CU02 (Registrar jugador), CU03 (Modificar datos de jugador), CU04 (Dar de baja)
Tipo de prueba	Funcional caja negra; pruebas de validación de entrada; pruebas de integridad de datos
Entorno	PC local — Java SE 17.0.10 — MySQL 8.0.36 — Windows 11 Pro 22H2
Datos de prueba	Base de datos inicializada con script datos_iniciales.sql; jugadores y usuarios de prueba
Criterio de aprobación	Resultado obtenido coincide con resultado esperado; sin errores de excepción no controlados
Responsable	Fonzo, Ignacio (desarrollador / tester)

5.2 Casos de Prueba

ID	CU	Descripción	Entrada	Resultado esperado	Estado
CP01	CU01	Login credenciales válidas (rol Entrenador)	con usuario="entrenador1", pass="rugby2026"	Acceso concedido. Menú de Entrenador habilitado	☐ Aprobado
CP02	CU01	Login contraseña incorrecta	con usuario="entrenador1", pass="wrongpass"	Mensaje de error: "Credenciales incorrectas". No se habilita menú	☐ Aprobado
CP03	CU01	Bloqueo tras 3 intentos fallidos	Tres intentos consecutivos con contraseña incorrecta	Cuenta bloqueada 5 min. Mensaje informativo al usuario	☐ Aprobado
CP04	CU02	Registro de jugador con datos completos y válidos	DNI: 45123456, División: M15, Posición: Pilar	Jugador creado con estado ACTIVO. Aparece en listado	☐ Aprobado
CP05	CU02	Registro con DNI duplicado	DNI ya existente: 45123456	Error: "El DNI ya está registrado". No se crea duplicado	☐ Aprobado

CP0 6	CU0 2	Registro con campos obligatorios vacíos	Nombre, Apellido o DNI en blanco	Error de validación por campo faltante. No se ejecuta INSERT	☐ Aprobado
CP0 7	CU0 3	Modificación de posición de jugador existente	id=3, nueva posición: "Hooker"	Datos actualizados en BD. Listado refleja el cambio	☐ Aprobado
CP0 8	CU0 4	Baja lógica de jugador activo	id=5, confirmar baja	Estado cambia a INACTIVO. No aparece en listas de selección de partido	☐ Aprobado

6. Definición de Base de Datos

6.1 Modelo Relacional Normalizado

El modelo relacional de SistRUGBY-SLTC fue diseñado aplicando las tres primeras formas normales (1FN, 2FN y 3FN) para garantizar integridad, eliminar redundancias y optimizar el rendimiento de las consultas (Date, 2004).

Forma Normal	Condición	Verificación en SistRUGBY-SLTC
1FN	Todos los atributos son atómicos; no hay grupos repetitivos ni atributos multivaluados	Cumple: cada celda contiene un único valor. Los eventos de partido se modelan como filas individuales en EVENTOS_PARTIDO, no como listas en PARTIDOS
2FN	Cumple 1FN y todos los atributos no clave dependen funcionalmente de la clave primaria completa	Cumple: todas las tablas tienen clave primaria simple (id_x AUTO_INCREMENT). No existen dependencias parciales
3FN	Cumple 2FN y no existen dependencias transitivas (atributo no clave que depende de otro atributo no clave)	Cumple: nombre del club rival está en CLUBES_RIVALES y se referencia por FK en PARTIDOS. La división se normaliza en tabla propia. Sin redundancias transitivas

Relaciones del modelo:

Tabla	Descripción	FK hacia	Cardinalidad
JUGADORES	Padrón de jugadores de todas las divisiones	DIVISIONES	N:1
PARTIDOS	Registro de encuentros deportivos por división y temporada	CLUBES_RIVALES, DIVISIONES	N:1 (c/u)
PLANTEL_PARTIDO	Asociación N:M entre jugadores y partidos con rol (titular/suplente)	PARTIDOS, JUGADORES	N:M resuelta
EVENTOS_PARTIDO	Registro de eventos durante el partido (try, tarjetas, etc.)	PARTIDOS, JUGADORES	N:1 (c/u)

6.2 Diagrama Entidad-Relación (DER)

El DER representa visualmente las entidades, sus atributos y las relaciones entre ellas. Las claves primarias se indican con el prefijo **PK** y las claves foráneas con el prefijo **FK**.

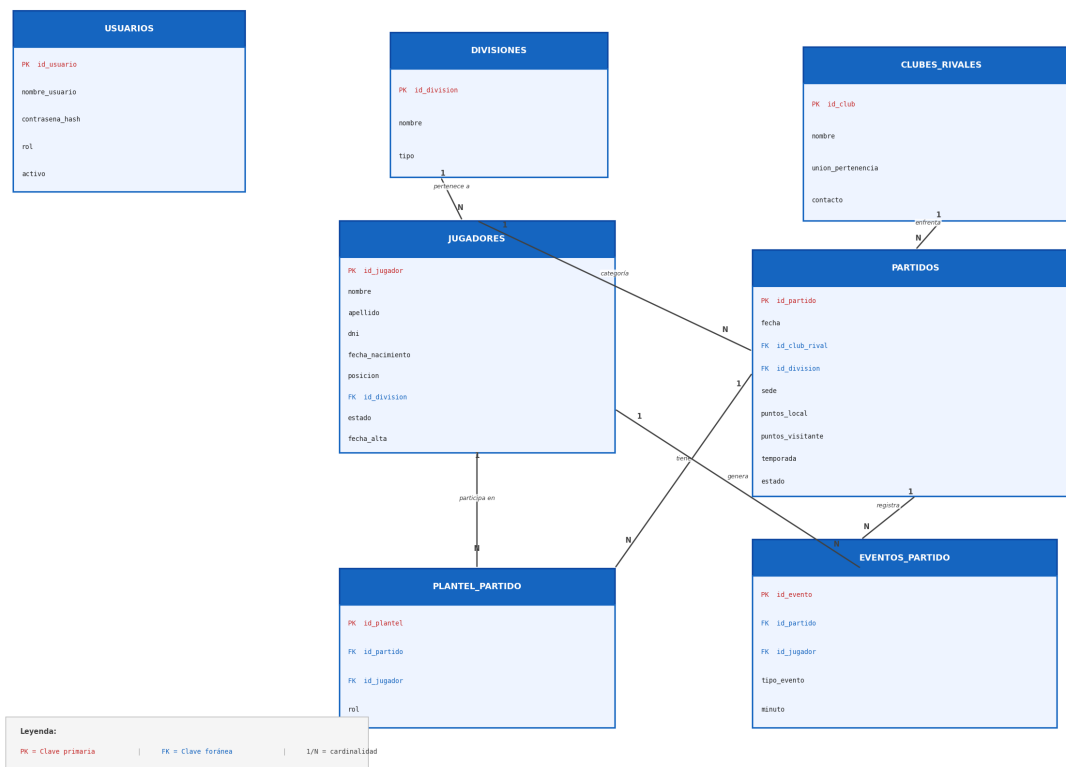


Figura 7 — Diagrama Entidad-Relación — SistRUGBY-SLTC — MySQL 8.0

6.3 Creación de Tablas (DDL)

A continuación se presentan los scripts SQL de creación de la base de datos y sus tablas, con restricciones de integridad referencial.

```

schema.sql
--
-- SistRUGBY-SLTC | schema.sql | MySQL 8.0
--
CREATE DATABASE IF NOT EXISTS sistrugby_sltc
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE sistrugby_sltc;

-- Tabla: USUARIOS
CREATE TABLE usuarios (
  id_usuario INT AUTO_INCREMENT PRIMARY KEY,
  nombre_usuario VARCHAR(50) NOT NULL UNIQUE,
  contrasena_hash VARCHAR(255) NOT NULL,
  rol ENUM('ADMINISTRADOR', 'ENTRENADOR', 'SECRETARIO') NOT NULL,
  activo TINYINT(1) DEFAULT 1
) ENGINE=InnoDB;

-- Tabla: DIVISIONES
CREATE TABLE divisiones (
  id_division INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE,
  tipo ENUM('JUVENIL', 'PLANTEL_SUPERIOR') NOT NULL
) ENGINE=InnoDB;

-- Tabla: JUGADORES
CREATE TABLE jugadores (
  id_jugador INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  apellido VARCHAR(100) NOT NULL,
  dni VARCHAR(15) NOT NULL UNIQUE,
  fecha_nacimiento DATE NOT NULL,
  posicion VARCHAR(50),
  id_division INT NOT NULL,
  estado ENUM('ACTIVO', 'INACTIVO') DEFAULT 'ACTIVO',
  fecha_alta DATE NOT NULL,
  CONSTRAINT fk_jug_div FOREIGN KEY (id_division) REFERENCES divisiones(id_division)
) ENGINE=InnoDB;

-- Tabla: CLUBES_RIVALES
CREATE TABLE clubes_rivales (
  id_club INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL UNIQUE,
  union_pertenencia VARCHAR(100),
  contacto VARCHAR(200)
) ENGINE=InnoDB;

-- Tabla: PARTIDOS
CREATE TABLE partidos (
  id_partido INT AUTO_INCREMENT PRIMARY KEY,
  fecha DATE NOT NULL,
  id_club_rival INT NOT NULL,
  id_division INT NOT NULL,
  sede ENUM('LOCAL', 'VISITANTE') NOT NULL,
  puntos_local INT DEFAULT 0,
  puntos_visitante INT DEFAULT 0,
  temporada YEAR NOT NULL,
  estado ENUM('PENDIENTE', 'FINALIZADO') DEFAULT 'PENDIENTE',
  CONSTRAINT fk_part_club FOREIGN KEY (id_club_rival) REFERENCES clubes_rivales(id_club),

```

```
CONSTRAINT fk_part_div FOREIGN KEY (id_division) REFERENCES divisiones(id_division)
) ENGINE=InnoDB;

-- Tabla: PLANTEL_PARTIDO (resuelve N:M Jugador - Partido)
CREATE TABLE plantel_partido (
id_plantel INT AUTO_INCREMENT PRIMARY KEY,
id_partido INT NOT NULL,
id_jugador INT NOT NULL,
rol ENUM('TITULAR','SUPLENTE') NOT NULL,
UNIQUE KEY uq_plantel (id_partido, id_jugador),
CONSTRAINT fk_pp_partido FOREIGN KEY (id_partido) REFERENCES partidos(id_partido),
CONSTRAINT fk_pp_jugador FOREIGN KEY (id_jugador) REFERENCES jugadores(id_jugador)
) ENGINE=InnoDB;

-- Tabla: EVENTOS_PARTIDO
CREATE TABLE eventos_partido (
id_evento INT AUTO_INCREMENT PRIMARY KEY,
id_partido INT NOT NULL,
id_jugador INT NOT NULL,
tipo_evento ENUM('TRY','CONVERSION','PENAL','DROP',
'TARJETA_AMARILLA','TARJETA_ROJA','SUSTITUCION') NOT NULL,
minuto INT,
CONSTRAINT fk_ev_partido FOREIGN KEY (id_partido) REFERENCES partidos(id_partido),
CONSTRAINT fk_ev_jugador FOREIGN KEY (id_jugador) REFERENCES jugadores(id_jugador)
) ENGINE=InnoDB;
```

6.4 Inserción de Registros (DML — INSERT)

```

datos_iniciales.sql
-- datos_iniciales.sql

-- Divisiones del sistema
INSERT INTO divisiones (nombre, tipo) VALUES
('M15', 'JUVENIL'), ('M16', 'JUVENIL'), ('M17', 'JUVENIL'),
('M18', 'JUVENIL'), ('M19', 'JUVENIL'),
('Pre-Intermedia', 'PLANTEL_SUPERIOR'),
('Intermedia', 'PLANTEL_SUPERIOR'),
('Primera', 'PLANTEL_SUPERIOR');

-- Clubes rivales de la USR
INSERT INTO clubes_rivales (nombre, union_pertenencia, contacto) VALUES
('Inti Rugby Club', 'USR', NULL),
('Quimilí Rugby Club', 'USR', NULL),
('Estudiantes Rugby', 'USR', NULL);

-- Usuarios del sistema (contraseñas: bcrypt hash de 'rugby2026')
INSERT INTO usuarios (nombre_usuario, contrasena_hash, rol) VALUES
('admin', '$2a$10$xyz...hashAdmin', 'ADMINISTRADOR'),
('entrenador1', '$2a$10$xyz...hashEnt', 'ENTRENADOR'),
('secretario1', '$2a$10$xyz...hashSec', 'SECRETARIO');

-- Jugadores (id_division=1 => M15, id_division=8 => Primera)
INSERT INTO jugadores
(nombre, apellido, dni, fecha_nacimiento, posicion, id_division, estado, fecha_alta)
VALUES
('Matías', 'González', '45123456', '2009-03-15', 'Pilar', 1, 'ACTIVO', '2026-03-01'),
('Lucas', 'Rodríguez', '44987654', '2008-07-22', 'Apertura', 1, 'ACTIVO', '2026-03-01'),
('Ignacio', 'Pérez', '43876543', '2007-11-10', 'Hooker', 2, 'ACTIVO', '2026-03-01'),
('Sebastián', 'López', '38001122', '2002-05-04', 'Octavo', 8, 'ACTIVO', '2026-03-01');

-- Partido de prueba: SLTC M15 vs Inti RC — 12/04/2026 — LOCAL — 21:14
INSERT INTO partidos
(fecha, id_club_rival, id_division, sede, puntos_local, puntos_visitante, temporada, estado)
VALUES ('2026-04-12', 1, 1, 'LOCAL', 21, 14, 2026, 'FINALIZADO');

-- Plantel del partido id=1
INSERT INTO plantel_partido (id_partido, id_jugador, rol) VALUES
(1, 1, 'TITULAR'), (1, 2, 'TITULAR'), (1, 3, 'SUPLENTE');

-- Eventos del partido id=1
INSERT INTO eventos_partido (id_partido, id_jugador, tipo_evento, minuto) VALUES
(1, 1, 'TRY', 12),
(1, 2, 'CONVERSION', 13),
(1, 1, 'TRY', 45),
(1, 2, 'PENAL', 60);

```

6.5 Consultas SQL (DML — SELECT)

Consulta 1 — Estadísticas individuales de un jugador por temporada

consulta_estadisticas.sql

```
SELECT
j.apellido,
j.nombre,
COUNT(DISTINCT pp.id_partido) AS partidos_jugados,
SUM(CASE WHEN e.tipo_evento = 'TRY' THEN 1 ELSE 0 END) AS tries,
SUM(CASE WHEN e.tipo_evento = 'CONVERSION' THEN 1 ELSE 0 END) AS conversiones,
SUM(CASE WHEN e.tipo_evento = 'PENAL' THEN 1 ELSE 0 END) AS penales,
SUM(CASE WHEN e.tipo_evento = 'DROP' THEN 1 ELSE 0 END) AS drops,
SUM(CASE WHEN e.tipo_evento = 'TARJETA_AMARILLA' THEN 1 ELSE 0 END) AS amarillas,
SUM(CASE WHEN e.tipo_evento = 'TARJETA_ROJA' THEN 1 ELSE 0 END) AS rojas,
-- Puntos Rugby: TRY=5, CONVERSION=2, PENAL=3, DROP=3
(SUM(CASE WHEN e.tipo_evento = 'TRY' THEN 5 ELSE 0 END) +
SUM(CASE WHEN e.tipo_evento = 'CONVERSION' THEN 2 ELSE 0 END) +
SUM(CASE WHEN e.tipo_evento = 'PENAL' THEN 3 ELSE 0 END) +
SUM(CASE WHEN e.tipo_evento = 'DROP' THEN 3 ELSE 0 END)) AS puntos_aportados
FROM jugadores j
LEFT JOIN plantel_partido pp ON j.id_jugador = pp.id_jugador
LEFT JOIN partidos p ON pp.id_partido = p.id_partido AND p.temporada = 2026
LEFT JOIN eventos_partido e ON e.id_jugador = j.id_jugador
AND e.id_partido = p.id_partido
WHERE j.id_jugador = 1
GROUP BY j.id_jugador, j.apellido, j.nombre;
```

Consulta 2 — Historial de partidos por división y temporada

consulta_historial.sql

```
SELECT
p.fecha,
cr.nombre AS rival,
p.sede,
p.puntos_local AS pts_local,
p.puntos_visitante AS pts_visita,
CASE
WHEN p.sede = 'LOCAL' AND p.puntos_local > p.puntos_visitante THEN 'VICTORIA'
WHEN p.sede = 'LOCAL' AND p.puntos_local < p.puntos_visitante THEN 'DERROTA'
WHEN p.sede = 'VISITANTE' AND p.puntos_local < p.puntos_visitante THEN 'VICTORIA'
WHEN p.sede = 'VISITANTE' AND p.puntos_local > p.puntos_visitante THEN 'DERROTA'
ELSE 'EMPATE'
END AS resultado
FROM partidos p
JOIN clubes_rivales cr ON p.id_club_rival = cr.id_club
WHERE p.id_division = 1 AND p.temporada = 2026
ORDER BY p.fecha;
```

Consulta 3 — Reporte completo de un partido (plantel + eventos)

consulta_reporte_partido.sql

```
-- Datos del partido
SELECT p.id_partido, p.fecha, cr.nombre AS rival, p.sede,
p.puntos_local, p.puntos_visitante, d.nombre AS division, p.temporada
FROM partidos p
JOIN clubes_rivales cr ON p.id_club_rival = cr.id_club
JOIN divisiones d ON p.id_division = d.id_division
WHERE p.id_partido = 1;

-- Plantel del partido
SELECT j.apellido, j.nombre, j.posicion, pp.rol
FROM plantel_partido pp
JOIN jugadores j ON pp.id_jugador = j.id_jugador
```

```
WHERE pp.id_partido = 1
ORDER BY pp.rol DESC, j.apellido;

-- Eventos del partido con jugador
SELECT e.minuto, e.tipo_evento,
CONCAT(j.apellido, ' ', j.nombre) AS jugador
FROM eventos_partido e
JOIN jugadores j ON e.id_jugador = j.id_jugador
WHERE e.id_partido = 1
ORDER BY e.minuto;
```

6.6 Eliminación de Registros (DML — DELETE / UPDATE)

El sistema privilegia la baja lógica (UPDATE de estado) sobre la eliminación física para preservar la integridad histórica. La baja física se reserva para datos incorrectamente ingresados.

```
operaciones_baja.sql
-- Baja LÓGICA de jugador (recomendada – preserva historial)
UPDATE jugadores
SET estado = 'INACTIVO'
WHERE id_jugador = 3;

-- Eliminación FÍSICA de un evento registrado por error
-- (permitida porque no afecta registros históricos de otros eventos)
DELETE FROM eventos_partido
WHERE id_evento = 5;

-- Eliminación de un plantel incompleto (partido aún no finalizado)
DELETE FROM plantel_partido
WHERE id_partido = 2;

-- Eliminación de un partido en estado PENDIENTE (no finalizado)
-- Se deben eliminar primero los registros dependientes (ON DELETE CASCADE no configurado)
DELETE FROM plantel_partido WHERE id_partido = 2;
DELETE FROM eventos_partido WHERE id_partido = 2;
DELETE FROM partidos WHERE id_partido = 2 AND estado = 'PENDIENTE';

-- NOTA: No se eliminan usuarios ni divisiones con registros asociados.
-- El sistema impide la eliminación física de entidades referenciadas por FK.
```


7. Definiciones de Comunicación

7.1 Entorno de Red e Infraestructura Física

SistRUGBY-SLTC opera en un entorno de red de área local (LAN) del club. La configuración mínima prevista es una única computadora que actúa como cliente y servidor simultáneamente (arquitectura monopuesto). Una expansión futura podría separar la PC cliente de un servidor MySQL dedicado en la misma LAN.

Componente	Especificación mínima	Rol en el sistema
PC del club	Intel Core i5 (6ª gen+), 8 GB RAM, 256 GB SSD, Windows 10/11 (64-bit)	Host de la JVM (aplicación) y del servidor MySQL
JRE 17+	Java Runtime Environment 17 o superior (OpenJDK o Oracle)	Entorno de ejecución de SistRUGBY-SLTC.jar
MySQL Server 8.0	MySQL Community Server 8.0.x en localhost, puerto TCP 3306	Motor de base de datos relacional; gestión de sistrugby_sltc
MySQL Connector/J	mysql-connector-j-8.3.0.jar incluido en el .jar ejecutable	Driver JDBC que provee la conectividad Java-MySQL
Red LAN (opcional)	Switch Ethernet 100/1000 Mbps, cable UTP Cat5e o Wi-Fi 802.11n	Infraestructura de red para expansión multiusuario futura

7.2 Protocolos y Estándares de Comunicación

La comunicación entre los componentes del sistema sigue el modelo de referencia OSI y TCP/IP. A continuación se detalla la pila de protocolos aplicada:

Capa OSI	Protocolo / Estándar	Rol en SistRUGBY-SLTC
7 — Aplicación	JDBC (Java Database Connectivity) — API estándar Java	Interfaz de alto nivel para envío de sentencias SQL desde Java hacia MySQL. Usa PreparedStatement para prevenir SQL Injection
6 — Presentación	MySQL Protocol — codificación UTF-8MB4	Serialización y codificación de los datos intercambiados entre driver JDBC y servidor MySQL
5 — Sesión	MySQL Session Management	Gestión de la conexión activa (handshake, autenticación SHA-256, keepalive)
4 — Transporte	TCP (Transmission Control Protocol) — Puerto 3306	Comunicación confiable y orientada a conexión entre la capa DAO (cliente) y MySQL Server. Garantiza entrega ordenada y sin errores

3 — Red	IP (Internet Protocol) — IPv4 127.0.0.1 (loopback)	Enrutamiento de paquetes entre capa de aplicación y servidor MySQL. En despliegue monopuesto se usa la dirección de loopback (localhost)
2 — Enlace de datos	IEEE 802.3 (Ethernet) / IEEE 802.11 (Wi-Fi)	Control de acceso al medio físico. En despliegue monopuesto esta capa no interviene (loopback). En LAN: Ethernet 100/1000 Mbps
1 — Física	Cable UTP Cat5e/6 o Wi-Fi 802.11n/ac	Medio físico de transmisión. Solo relevante en configuración de red LAN con servidor MySQL separado de la aplicación

Control de integridad transaccional: Las operaciones que involucran múltiples tablas (por ejemplo, registrar un partido + plantel + eventos) se ejecutan en una única transacción JDBC mediante `conn.setAutoCommit(false)`, `conn.commit()` y `conn.rollback()`, garantizando las propiedades ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad).

Seguridad en la comunicación: Para la configuración de escritorio monopuesto el tráfico no abandona el host (loopback), por lo que no se requiere cifrado TLS. Si se migra a servidor dedicado, se recomienda activar SSL/TLS en la conexión JDBC mediante el parámetro `useSSL=true` en la URL de conexión.

8. Referencias Bibliográficas

- Date, C. J. (2004). Introducción a los sistemas de bases de datos (8.^a ed.). Pearson Prentice Hall.
- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.
- Kendall, K., & Kendall, J. (2011). Análisis y diseño de sistemas (8.^a ed.). Pearson Education.
- Larman, C. (2003). UML y Patrones: Introducción al análisis y diseño orientado a objetos y al proceso unificado (2.^a ed.). Pearson Prentice Hall.
- MySQL AB. (2024). MySQL 8.0 Reference Manual. Recuperado de <https://dev.mysql.com/doc/refman/8.0/en/>
- Oracle Corporation. (2024). Java SE 17 Documentation. Recuperado de <https://docs.oracle.com/en/java/javase/17/>
- Oracle Corporation. (2024). JDBC API Specification. Recuperado de <https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/module-summary.html>
- Pressman, R. S. (2010). Ingeniería del software: un enfoque práctico (7.^a ed.). McGraw-Hill.